

Wowlo

Tuition Management App

FULL PRODUCT DOCUMENT

Version 2.0 — Updated & Finalised

Laravel · Blade · Tailwind CSS · Alpine.js · PostgreSQL · PWA

Table of Contents

1. Idea & Requirements	2
2. User Flow	5
3. Wireframe	7
4. Design (UI / UX Decisions)	12
5. Tech Stack & Architecture Decisions	15
6. Database Planning	17
7. Code Architecture	23
8. Testing	26
9. Deployment	27
10. Improvement & Phased Growth	29

◆ **UPDATED:** This is Version 2.0 of the product document. Key updates: homeworks table added, homework attachments moved to Phase 1 MVP, Alpine.js added, Google OAuth confirmed with google_id column, fee unlock via global .env password, hosting stack updated to Render + Neon + UptimeRobot.

1 Idea & Requirements

Problem Statement

This app centralises homework assignments and tutor–student communication in one secure platform, so tutors, students and parents can clearly see assigned homework, main details and messages anytime without confusion. It also eliminates manual tuition fee calculation and WhatsApp billing.

What problem does the app solve?

- Tutors manage homework via WhatsApp/In Person — messages get buried
- Parents/students forget homework
- No central place to see assigned homework or read tutor messages
- Tutor wastes time repeating same instructions
- Tutor cannot remember each student's learning progress
- Calculating tuition fees manually and sending individually via WhatsApp is time-consuming

■■ **IMPORTANT:** Only doing MVP for now. To be done in phases.

Who Are the Users?

User Types (Phase 1 — MVP)

1. Student / Parent (same login role)

- Logs in
- Sees: Profile, Assigned Homework, Tutor Messages, Tuition Fees & Details, Outstanding Payment, Remarks
- Cannot see other students' data

2. Tutor / Admin

- Creates homework (with attachments)
- Assigns homework to students
- Sends messages via in-app inbox
- Manages users (students)
- Notifies for payment collection / outstanding payment
- Manages tuition schedule

Core Features (MVP — Must Have)

Student / Parent Features

- Login (Email/Password + Google OAuth)
- Dashboard (Summary)

- Homework list & details
- Messages from tutor (read-only)
- Payment / Outstanding Payment (hidden by default — unlocked via parent password)
- Profile (name, address, email, phone numbers)
- Mark homework as Done / Not Done
- View & download past year exam papers (filter by subject / year)
- Receive push notifications (via PWA) for new homework and messages
- Take quizzes assigned by tutor (MCQ only in Phase 1)
- View quiz results and marks

Tutor / Admin Features

- Login (Email/Password + Google OAuth)
- Create student accounts
- Assign homework with title, subject, description, due date, attached files
- Send messages to individual students
- View homework status per student
- Manage tuition fees & record payments
- Upload past year exam papers (filterable by subject and year)
- WhatsApp billing message generator (generate payment request message to copy & send to parents)
- Create and assign quizzes to students (MCQ only in Phase 1, filtered by subject/level/topic/exam type)

Features NOT in MVP (Future Phases)

- Attendance tracking (Phase 2)
- Exams & Report Cards — auto-generated (Phase 2)
- Short Answer Quiz with AI marking (Phase 2)
- SMS
- Reports & Backup/Export
- Staff salary
- Batch management
- Parent–student separate roles
- Message replies (students)
- Edit profile (student self-edit)
- PWA offline mode (Phase 2)
- Native mobile app — React Native (Phase 3, only if needed)

Platform Scope

Phase	Scope
Phase 1 (Now)	Wowlo Web App — Laravel + Blade + Tailwind CSS + Alpine.js + PWA

PWA (Phase 1)	Progressive Web App — installable on phone, push notifications for parents/students
Responsive	Mobile-responsive design (parents mainly use phone browser)
Phase 2+	PWA enhancements — offline mode, advanced caching
Phase 3 (Future)	Native mobile app (React Native) — only if genuinely needed

■ **App Goal:** A secure tuition management web app where students log in to view assigned homework and tutor messages, and tutors can manage homework assignments and payment collections efficiently.

2 User Flow

Purpose: Defines exactly how users move through the app. This makes wireframing, database design, and route planning straightforward.

Authentication Flow (All Users)

- User lands on Login Page
 - Enters Email + Password (or clicks Google OAuth)
 - System verifies credentials
 - Redirect based on role: Student → Student Dashboard, Tutor → Tutor Dashboard
 - If invalid → show error message
- No public registration — accounts are created by the tutor/admin only.

Student / Parent User Flow

Dashboard (Landing Page)

After login, student sees:

- Upcoming homework (next due date)
- Latest tutor message
- Quick navigation menu
- Outstanding payment summary

Homework Flow

- **Step 1:** Click Homework
- **Step 2:** See Homework List — Title, Subject, Due Date, Status (Pending / Done)
- **Step 3:** Click a homework item
- **Step 4:** See Homework Details — Full description, due date, attachments, start date
- **Step 5:** Mark homework as Done / Not Done

Messages Flow

- Click Messages
- See Inbox — list of messages from tutor
- Click message
- Read message details
- Cannot reply (one-way in MVP)

Profile Flow

- Click Profile
- View: Name, Parent's Name, Email, Phone numbers (Student / Father / Mother / NOK / Home), Address
- Confidentiality notice displayed
- Edit profile — Phase 2

Tuition Fee Flow

- Only accessible after parent enters the special fee unlock password
- View: Monthly fee, payment history, outstanding balance
- Cannot edit — read only for parents

Tutor / Admin User Flow

Dashboard (Landing Page)

- Total students count
- Pending homework count
- Recent activity log

Student Management Flow

- **Step 1:** Click Students
- **Step 2:** See Student List
- **Step 3:** Click Add Student
- **Step 4:** Enter details: Name, Email, Password, Contact details, Address
- **Step 5:** Save → Student account created
- **Step 6:** Edit → Update student details
- **Step 7:** Delete → Remove student profile

Homework Management Flow

- **Step 1:** Click Homework
- **Step 2:** See Homework List
- **Step 3:** Click Create Homework
- **Step 4:** Enter: Title, Subject, Description, Due Date, Assigned Student, Attachment (optional)
- **Step 5:** Save → Homework visible to assigned student immediately

Messaging Flow

- **Step 1:** Click Messages
- **Step 2:** Click Compose Message
- **Step 3:** Select student
- **Step 4:** Write subject + message body
- **Step 5:** Send → Message appears in student inbox

Key Rules

- Students only see their own data • Tutors see all students • No batch logic in MVP • No in-app payment processing — only fee viewing and recording • No WhatsApp/SMS integration in MVP

3 Wireframe

Purpose: Defines the skeleton of every page — layout, elements, and purpose. No styling decisions at this stage.

Global Layout (All Authenticated Pages)

- Top Navigation / Header: App name, logged-in user name, logout button
- Side Navigation (role-based): menu items change based on user role
- Main Content Area: page-specific content

Landing Page (Public — Homepage)

MVP: Simple landing page. Full marketing site (About Us, Courses, Contact) in Phase 2.

- Wowlo logo + tagline (warm, encouraging)
- Hero section: headline + short description + Login button
- Brief features overview (3 key benefits)
- Footer: Privacy Policy link
- Navigation: Login button only in MVP

Login Page (Public)

- Wowlo logo
- Email input
- Password input
- Google OAuth login button
- Login button
- Error message area
- Forgot Password link
- Back to Home link

Privacy Policy Page (Public)

- Required for PDPA compliance (Singapore)
- What data is collected and why
- How data is stored and protected
- Contact information for data queries

Student / Parent Pages

Student Dashboard

- Welcome message

- Upcoming homework (max 3)
- Latest tutor message
- Navigation: Homework, Messages, Profile, Tuition Fee & Details

Homework List Page

- Page title: My Homework
- Homework table: Title, Subject, Due Date, Status
- Clickable homework items

Homework Detail Page

- Homework title, Subject
- Description (long text)
- Start date / Due date
- Status indicator
- Attachment file (if any)
- Mark as Done button

Messages Inbox Page

- Page title: Messages
- Message list: Subject/preview, Date
- Clickable message items

Message Detail Page

- Message subject
- Sender (Tutor)
- Date sent
- Full message body

Profile Page

- Name, Parent Name, Email
- Phone numbers (Student, Father, Mother, NOK, Home)
- Address
- Confidentiality notice
- Read-only in MVP

Tuition Fee Page

- Password-protected entry
- Monthly fee amount
- Payment history
- Outstanding balance (calculated)

Exam Papers Page (Student/Parent)

- Filter by Subject dropdown
- Filter by Year dropdown
- Papers list: Title, Subject, Year, Download button
- All students see all papers

Quiz List Page (Student)

- List of assigned quizzes
- Status: Not Started / In Progress / Completed
- Score shown after completion

Take Quiz Page (Student)

- One question at a time or all questions on one page
- MCQ: select A/B/C/D
- Submit button
- Cannot change answers after submit

Quiz Result Page (Student)

- Total score: X / Y marks
- Per question: correct/wrong indicator
- Corrections section: student can write corrections on-screen for wrong answers

PWA Install Prompt

- Shown once after first login only
- Banner: Add Wowlo to your home screen for a better experience!
- Install button + Dismiss button
- Never shown again after dismissed
- Uses browser beforeinstallprompt event

Tutor / Admin Pages

Tutor Dashboard

- Total students count
- Pending homework count
- Recent homework created
- Navigation: Students, Homework, Messages, Homework Status, Finance

Student List Page

- Page title: Students
- Student table: Name, Email
- Add Student button

- Clickable student rows

Add / Edit Student Page

- Name input
- Email input
- Password input
- Phone number inputs (x5: Student, Mother, Father, NOK, Home)
- Address input
- Save button

Homework List (Tutor)

- Homework table: Title, Student, Due Date, Status
- Sort & filter by: Title, Student, Due Date, Status
- Delete homework button
- Create Homework button

Create / Edit Homework Page

- Title input
- Subject input
- Description textarea
- Due date picker
- Student dropdown
- Attachment upload
- Save button

Messages Page (Tutor)

- Sent message list
- Sort/filter message list
- Compose Message button

Compose Message Page

- Student dropdown
- Subject input
- Message textarea
- Send button

Homework Status Page

- Student dropdown
- Homework list with status: Pending / Done

Finance Page

- Student dropdown
- Tuition fee details
- Record payment form
- Payment history
- Outstanding balance

Exam Papers Page (Tutor)

- Upload paper form: Title, Subject, Year, File upload, Remarks
- List of all uploaded papers
- Delete paper button

Quiz List Page (Tutor)

- All quizzes created
- Create Quiz button
- View results per student

Create Quiz Page (Tutor)

- Title, Subject, Level, Topic, Exam Type dropdowns
- Add questions section
- MCQ: question text, 4 options, correct answer, marks
- Save and publish button

WhatsApp Billing Page (Tutor)

- Select student dropdown
- Auto-fills: student name, amount due, month, payment method (PayNow)
- Generated message preview
- Copy to clipboard button

4 Design (UI / UX Decisions)

Design Goals

Simple · Clean · Calm · No distractions · Easy for parents & students (non-tech users) ■
Think: tuition parents, not developers.

Target Users (Design Perspective)

User	Design Implication
Parents	30–55 years old. Need clarity and simplicity.
Students	Primary / Secondary school age. Simple UI.
Tutor	Daily heavy usage. Needs efficiency.

Design Implications

- Large readable text
- Clear, large buttons
- Minimal icons — use text labels
- No fancy animations

Colour Scheme

◆ **UPDATED:** Updated to match Wowlo brand: warm, encouraging and connected vibe.

Colour Role	Purpose & Reasoning
Primary — Warm Violet / Purple	Creative, educational, inspiring — feels like growth and learning
Background — Soft Cream / Off-white	Warmer than pure white, easier on eyes for long study sessions
Accent — Amber / Golden Yellow	Achievement, motivation, encouragement — reward feeling
Success — Soft Green	Homework done, payment confirmed states
Error — Soft Red	Error messages only

■ Reference apps using similar warm palette: Duolingo, Notion, Khan Academy — all educational and warm-feeling.

Typography Rules

◆ **UPDATED:** Font updated to Nunito — rounded, friendly and warm with real personality.
Perfect for Wowlo's encouraging vibe. Used by many educational apps.

- Font: Nunito (Google Fonts — free)
- Headings: Nunito Bold — warm, clear, strong
- Body text: Nunito Regular — easy to read for long sessions
- Labels/captions: Nunito Light — subtle, clean

- No decorative fonts — readability is priority for parents and young students

Layout Rules

- Cards-based dashboard layout — each card = one purpose
- One-column forms — clear labels above inputs
- Big clickable buttons (parents using one hand on phone)
- Fixed sidebar on desktop, collapsible drawer on mobile
- Tables become stacked cards on mobile
- Full-width buttons on mobile

Status & Feedback Design

Element	Design
Homework Pending	Grey / Orange indicator
Homework Done	Green indicator
Message Unread	Bold text
Message Read	Normal text weight
Error states	Red text, simple plain language

Accessibility

- High contrast text
- Clickable areas not too small
- No reliance on colour alone — use text + colour for status
- Enlarged elements on hover

App Logo ♦ Phase 1 MVP

■ PHASE 1 MVP — Official Wowlo logo to be used across the app, landing page, PWA icon, and all branding.



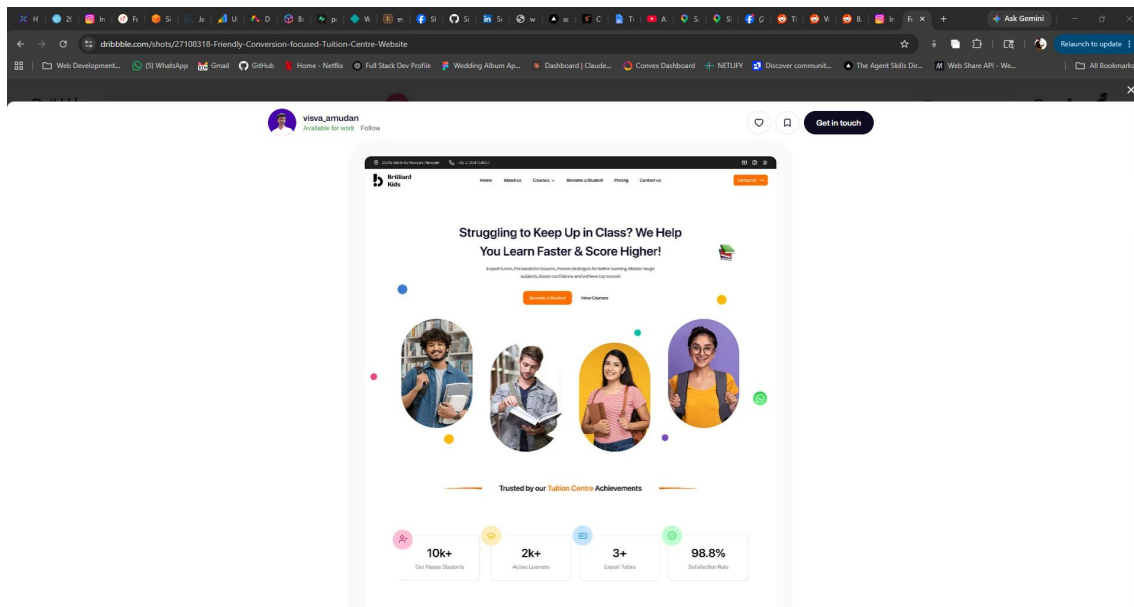
Wowlo Official Logo

■■ IMPORTANT LOGO NOTE: The square grid lines visible in the logo image above are the design canvas/grid from the design tool — they are NOT part of the actual logo. When using this logo: Export logo on a transparent background (PNG) — grids will disappear Ask designer to export final version without the grid Final logo = red icon + WOWLO text + SINCE 2026 text only Background should be transparent for use on any colour surface

Design Reference ♦ Phase 1 MVP

■ PHASE 1 MVP — This design reference applies to the Wowlo landing page and overall app UI for Phase 1.

Reference design that matches Wowlo's warm, friendly and educational vibe. Source: <https://dribbble.com/shots/27108318-Friendly-Conversion-focused-Tuition-Centre-Website>
Key design elements to take inspiration from: Warm colour palette with colourful accent dots
Clean hero section with strong headline and CTA buttons Friendly student imagery Stats section for credibility (students, satisfaction rate) Navigation: Home, About Us, Courses, Login



Design Reference Screenshot — Dribbble by visva_amudan

5 Tech Stack & Architecture Decisions

◆ **UPDATED:** Tech stack finalised after thorough evaluation. Key decisions: Blade over React (simpler for MVP dashboard), Alpine.js for interactivity, Neon (PostgreSQL) over MySQL, Render for hosting.

Final Tech Stack

Component	Choice	Reason
Backend	Laravel (PHP)	Built-in auth, MVC, security defaults, perfect for CRUD apps
Frontend	Laravel Blade	Server-side rendering, faster MVP, no API layer needed
Styling	Tailwind CSS	Fast prototyping, consistent design, mobile-first utilities
Interactivity	Alpine.js	Lightweight (15kb), Blade-native, handles modals/toggles with
Database	PostgreSQL (Neon)	Free generous tier, scale-to-zero, works perfectly with Laravel
Authentication	Laravel Breeze + Socialite	Session auth + Google OAuth in one setup
PWA	Service Worker + Web Manifest	Makes app installable on phone, enables push notifications
Push Notifications	laravel-notification-channels/webpush	Notifies parents/students on new homework and messages
App Hosting	Render (free tier)	Easy Laravel deploy, free, upgradeable to \$7/mo when needed
Keep-Alive	UptimeRobot	Pings app every 5 min to prevent Render cold starts — free
DB Hosting	Neon	100 free projects, 0.5GB each, scale-to-zero, never deletes or

Why Blade Over React?

This is a dashboard/admin tool — not a flashy consumer app. Parents and students care about clarity and ease of use, not animations. • Blade + Tailwind can achieve all required UI patterns (cards, tables, forms, sidebars) • Alpine.js handles all small interactive needs (mark done, dropdowns, modals) without React complexity • One codebase, one deployment, faster MVP • Can always migrate to React later — Laravel backend stays untouched

Why Neon Over MySQL / Supabase?

Neon ■ (Chosen)	Supabase	MySQL (Old)
Neon (PostgreSQL)	Supabase	MySQL (local)
100 free projects	2 free projects	Local only (XAMPP)
0.5GB per project	500MB total	N/A
Scale-to-zero — never billed for idle	Pauses after 7 days idle	N/A
No deletion on free tier	No deletion on free tier	N/A
Works with Laravel Eloquent	Works with Laravel	Works with Laravel
■ Best for irregular traffic	Extra features not needed	Not suitable for production

Hosting Setup

Component	Service
Laravel App	Render — free tier (spins down after 15 min idle)
Keep-Alive	UptimeRobot — pings app every 5 min, prevents cold starts. Free forever for personal use.
Database	Neon — hosted PostgreSQL, free tier, scale-to-zero
Cost Now	\$0/month — completely free
Cost Later	\$7/month for Render Starter (no cold starts) when real students use it daily

Environment & Tooling

- PHP 8+
- Composer
- Laravel Artisan
- Git + GitHub (private repo)
- XAMPP for local development

Security (Laravel Defaults)

- CSRF protection on all forms
- Password hashing via bcrypt
- Route protection via middleware
- Role-based access control (student vs tutor)
- Environment variables via .env (never committed to Git)
- Google OAuth via Laravel Socialite

6 Database Planning

◆ **UPDATED:** homeworks table columns now fully defined. Homework attachments moved to Phase 1 MVP. google_id column added to users table. Database changed from MySQL to PostgreSQL (Neon).

Database: PostgreSQL hosted on Neon **ORM:** Laravel Eloquent **Tables:** 6 core tables (lean, not bloated)

Core Tables (MVP — Final)

- users
- homeworks ◆ (fully defined — updated)
- homework_statuses
- messages
- tuition_fees
- payments
- exam_papers ◆ (new)
- push_subscriptions ◆ (new — PWA)
- quizzes ◆ (new)
- quiz_questions ◆ (new)
- quiz_attempts ◆ (new)
- quiz_answers ◆ (new)

1. users Table

Stores both students and tutors, differentiated by role column.

Column	Type	Notes
id	Primary Key	Auto-increment
name	string	Full name
email	string (unique)	Login email
password	string (hashed)	bcrypt hashed
google_id	string (nullable) ◆	Google OAuth ID — required for Google login
role	enum: student tutor	Role-based access
phone_1	string (nullable)	Student's own number
phone_2	string (nullable)	Father's number
phone_3	string (nullable)	Mother's number
phone_4	string (nullable)	Next of kin
phone_5	string (nullable)	Home phone (optional)
address	text (nullable)	Home address
created_at	timestamp	Auto
updated_at	timestamp	Auto

◆ google_id is nullable — only populated when user logs in via Google OAuth. Required for Socialite to identify returning Google users.

■■■ At least one phone number must be filled. If none provided, throw a validation error.

2. homeworks Table ◆ (Phase 1 MVP)

◆ **UPDATED:** This table was missing from the original document. Now fully defined.
Homework attachment is Phase 1 MVP.

Column	Type	Notes
id	Primary Key	Auto-increment
tutor_id	FK → users.id	The tutor who created the homework
student_id	FK → users.id	The student it's assigned to
title	string	Homework title
subject	string	Subject (e.g. Maths, English)
description	text	Full homework instructions
due_date	date	When homework is due
attachment_path	string (nullable)	File path in Laravel storage — Phase 1 MVP
created_at	timestamp	Auto (used as 'given date')
updated_at	timestamp	Auto

3. homework_statuses Table

Column	Type	Notes
id	Primary Key	Auto-increment
homework_id	FK → homeworks.id	
student_id	FK → users.id	
status	enum: pending done	Default: pending
updated_at	timestamp	Auto

4. messages Table

Column	Type	Notes
id	Primary Key	
sender_id	FK → users.id	Tutor
receiver_id	FK → users.id	Student/Parent
subject	string	Message subject
body	text	Full message content
is_read	boolean	Default: false
created_at	timestamp	Auto

5. tuition_fees Table

Defines the expected fee structure per student.

Column	Type	Notes
id	Primary Key	
student_id	FK → users.id	One fee structure per student
monthly_fee	decimal(8,2)	Expected monthly amount
currency	string	Default: SGD
billing_cycle	enum: monthly weekly	
start_date	date	When billing started
remarks	text (nullable)	Any special notes
created_at	timestamp	Auto
updated_at	timestamp	Auto

6. payments Table

Records actual payments made by students/parents.

Column	Type	Notes
id	Primary Key	
student_id	FK → users.id	
amount_paid	decimal(8,2)	Actual amount paid
payment_date	date	
payment_method	enum: cash bank_transfer paynow paypal	
remarks	text (nullable)	Any notes
created_at	timestamp	Auto

9. quizzes Table ♦ (Phase 1 MVP — New)

♦ **UPDATED:** Tutor creates quizzes filtered by subject, level, topic and Singapore exam schedule (WA1 / Mid Year / WA2 / End Year). Phase 1: MCQ only. Phase 2: Short Answer with AI marking.

Column	Type	Notes
id	Primary Key	Auto-increment
tutor_id	FK → users.id	The tutor who created the quiz
title	string	e.g. P4 Science WA1 Chapter 3
subject	string	e.g. Science, Maths, English
level	string	e.g. Primary 4, Primary 5
topic	string	e.g. Photosynthesis, Fractions
exam_type	enum: WA1 MidYear WA2 EndYear	Singapore primary school exam schedule
created_at	timestamp	Auto
updated_at	timestamp	Auto

10. quiz_questions Table ♦ (Phase 1 MVP — New)

Column	Type	Notes
id	Primary Key	
quiz_id	FK → quizzes.id	
question_text	text	The question
question_type	enum: mcq short_answer	Phase 1: mcq only. Phase 2: short_answer
option_a	string (nullable)	MCQ option A
option_b	string (nullable)	MCQ option B
option_c	string (nullable)	MCQ option C
option_d	string (nullable)	MCQ option D
correct_answer	string	MCQ: A/B/C/D. Short answer: full correct answer text
marks	integer	Marks for this question
order	integer	Display order in quiz
created_at	timestamp	Auto

11. quiz_attempts Table ♦ (Phase 1 MVP — New)

Column	Type	Notes
id	Primary Key	
quiz_id	FK → quizzes.id	
student_id	FK → users.id	
total_marks	integer	Total possible marks
obtained_marks	integer	Marks student scored
completed_at	timestamp (nullable)	Null if still in progress
created_at	timestamp	Auto

12. quiz_answers Table ♦ (Phase 1 MVP — New)

Column	Type	Notes
id	Primary Key	
attempt_id	FK → quiz_attempts.id	
question_id	FK → quiz_questions.id	
student_answer	text	What the student answered
is_correct	boolean	MCQ: auto-checked. Short answer: AI-checked (Phase 2)
marks_awarded	integer	0 or full marks per question
ai_feedback	text (nullable)	Phase 2 only — AI marking explanation
created_at	timestamp	Auto

■ Phase 1 — MCQ only: is_correct checked by comparing student_answer to correct_answer directly. No AI needed.

■ Phase 2 — Short Answer: send question + correct_answer + student_answer to Google Gemini API (free tier, 1500 calls/day). Fallback: Claude Haiku API (~\$10/bin/sh.001 per question). Store AI feedback in ai_feedback column.

■ Corrections: After completing quiz, student can review wrong answers on-screen and write corrections.

7. exam_papers Table ♦ (Phase 1 MVP — New)

♦ **UPDATED:** New feature: Tutor uploads past year exam papers. Parents and students can view and download. Filterable by subject and year.

Column	Type	Notes
id	Primary Key	Auto-increment
tutor_id	FK → users.id	The tutor who uploaded the paper
title	string	e.g. 2023 PSLE Maths Paper 1
subject	string	e.g. Maths, English, Science
year	integer	e.g. 2023, 2022 — used for filtering
file_path	string	Path in Laravel storage
remarks	text (nullable)	Optional notes about the paper
created_at	timestamp	Auto
updated_at	timestamp	Auto

■ Access: All students and parents can view ALL exam papers (not student-specific). Tutor manages uploads.

■ Filters: By subject (e.g. Maths) and/or year (e.g. 2023). Both filters can be combined.

8. push_subscriptions Table ♦ (Phase 1 MVP — PWA)

♦ **UPDATED:** Required for PWA push notifications. Stores each device/browser subscription endpoint so Laravel can send push notifications to parents and students.

Column	Type	Notes
id	Primary Key	Auto-increment
user_id	FK → users.id	The user who subscribed
endpoint	text	Browser push endpoint URL
public_key	text	VAPID public key for this subscription
auth_token	text	Auth token for this subscription
created_at	timestamp	Auto
updated_at	timestamp	Auto

■ Package: laravel-notification-channels/webpush handles this automatically.

■ VAPID keys stored in .env: VAPID_PUBLIC_KEY and VAPID_PRIVATE_KEY.

■ When to notify: New homework assigned, new message received.

Outstanding Payment — Key Design Decision

■■ Outstanding payment is NOT a stored column. It is CALCULATED dynamically. Formula: Outstanding = (Total Tuition Fee Due) - (Total Payments Made) Why: Prevents data inconsistency, no manual syncing needed, always accurate, industry best practice. In Laravel: calculated using Eloquent queries, shown only to parents/tutors.

Fee Unlock — Design Decision

❖ **UPDATED:** The tuition fee section is hidden from students by default. To unlock it, the parent enters a global password. Implementation: Store as FEE_VIEW_PASSWORD in .env file (e.g. FEE_VIEW_PASSWORD=tuition2025) Why global (not per-student): Simpler for MVP. Data is already restricted per student by role. The password just prevents students from seeing fees on a shared device. Phase 2 upgrade: Per-student unlock password if needed.

Relationships Summary

Model	Relationship	Related Model
User (Student)	hasMany	Homeworks
User (Student)	hasMany	HomeworkStatuses
User (Student)	hasMany	Payments
User (Student)	hasOne	TuitionFee
User (Student)	hasMany	Messages (received)
User (Tutor)	hasMany	Homeworks (created)
User (Tutor)	hasMany	Messages (sent)
TuitionFee	belongsTo	User (Student)
Payment	belongsTo	User (Student)
Homework	belongsTo	User (Student) + User (Tutor)
Homework	hasOne	HomeworkStatus
User (Tutor)	hasMany	ExamPapers (uploaded)
ExamPaper	belongsTo	User (Tutor)
User	hasMany	PushSubscriptions
PushSubscription	belongsTo	User
User (Tutor)	hasMany	Quizzes (created)
User (Student)	hasMany	QuizAttempts
Quiz	hasMany	QuizQuestions
Quiz	hasMany	QuizAttempts
QuizAttempt	hasMany	QuizAnswers
QuizQuestion	hasMany	QuizAnswers

Security & Visibility Rules

Action	Student	Parent	Tutor
View homework	■	■	■
Create homework	■	■	■
View fees	■ (hidden)	■ (with password)	■
Record payment	■	■	■
Manage students	■	■	■
Send messages	■	■	■

7 Code Architecture

Follows standard Laravel MVC: Models → Controllers → Views → Routes → Middleware. No shortcuts, no spaghetti code.

Models

One model per database table.

Model File	Responsibility
app/Models/User.php	Auth, role checking, all relationships
app/Models/Homework.php	Homework data, belongs to student + tutor, has status
app/Models/HomeworkStatus.php	Tracks completion, belongs to homework + student
app/Models/Message.php	Tutor→Student comms, read/unread tracking
app/Models/TuitionFee.php	Expected monthly fee per student
app/Models/Payment.php	Actual payments, used to calculate outstanding
app/Models/ExamPaper.php	Past year exam papers — uploaded by tutor, viewed by all
app/Models/PushSubscription.php	PWA push notification subscriptions per user device
app/Models/Quiz.php	Quiz — belongs to tutor, has many questions and attempts
app/Models/QuizQuestion.php	Individual question — MCQ options + correct answer stored here
app/Models/QuizAttempt.php	Student attempt — stores total and obtained marks
app/Models/QuizAnswer.php	Student answer per question — is_correct, marks awarded, AI feedback (Phase 2)

Controllers

Thin controllers — logic here, not in views or models.

Controller File	Responsibility
Auth/LoginController.php	Email/password + Google OAuth login, logout
DashboardController.php	Load role-based dashboard data
StudentController.php	Tutor only — create/edit/delete student accounts
HomeworkController.php	Tutor creates/assigns; Student views/marks done
MessageController.php	Tutor sends; Student reads
TuitionFeeController.php	Tutor sets fee; Parent views with password

PaymentController.php	Tutor records payments; Parent views history + outstanding
ExamPaperController.php	Tutor uploads papers; Students/parents view and download; filter by subject + year
PushSubscriptionController.php	Handles PWA push subscription registration and notification dispatch
QuizController.php	Tutor creates quizzes + questions; Student takes quiz; Marks calculated automatically
WhatsAppBillingController.php	Generates formatted WhatsApp payment request message for tutor to copy and send to parent

Routes Structure

All routes in routes/web.php, grouped by role with middleware protection.

Route Groups: /login, /logout — public /student/* — auth + role:student middleware /tutor/* — auth + role:tutor middleware ■ Students CANNOT access tutor routes. ■ Tutors CAN access student data for management.

Middleware

Middleware	Purpose
RoleMiddleware.php	Checks user role before allowing route access. Prevents URL hacking.
FeePasswordMiddleware.php	Checks if parent has entered the fee unlock password for this session.

Blade Views Structure

Views organised by role, sharing one common layout.

```
resources/views/
├── layouts/
│   └── app.blade.php      ← Header + Sidebar + Content slot
├── auth/
│   └── login.blade.php
├── public/
│   ├── landing.blade.php
│   ├── privacy.blade.php
│   └── student/
│       ├── dashboard.blade.php
│       ├── homework/
│       │   ├── index.blade.php
│       │   ├── show.blade.php
│       │   └── messages/
│       │       ├── index.blade.php
│       │       └── show.blade.php
│       ├── exam-papers/
│       │   └── index.blade.php
│       └── quiz/
```



```

■ ■ ■■■ index.blade.php
■ ■ ■■■ take.blade.php
■ ■ ■■■ result.blade.php
■ ■■■ finance/
■ ■ ■■■ index.blade.php
■ ■■■ profile.blade.php
■■■ tutor/
    ■■■ dashboard.blade.php
    ■■■ students/
        ■ ■■■ index.blade.php
        ■ ■■■ create.blade.php
        ■ ■■■ edit.blade.php
    ■■■ homework/
        ■ ■■■ index.blade.php
        ■ ■■■ create.blade.php
        ■ ■■■ edit.blade.php
    ■■■ messages/
        ■ ■■■ index.blade.php
        ■ ■■■ compose.blade.php
    ■■■ exam-papers/
        ■ ■■■ index.blade.php
    ■■■ quiz/
        ■ ■■■ index.blade.php
        ■ ■■■ create.blade.php
    ■■■ whatsapp-billing/
        ■ ■■■ index.blade.php
    ■■■ finance/
        ■■■ index.blade.php

```

Alpine.js Usage

◆ **UPDATED:** Alpine.js is added for lightweight interactivity — no need for React for these use cases.

Feature	Alpine.js Use
Mark homework Done/Not Done	Toggle without full page reload
Sidebar mobile toggle	Show/hide sidebar on mobile
Modal popups	Confirm delete, fee password entry
Message read toggle	Mark as read without reload
Form validation feedback	Instant input validation UI

Business Logic Rules

- Controllers do logic — not views, not models
- Models handle data access and relationships
- Views only handle display
- No raw SQL in controllers — use Eloquent
- No logic in Blade views — keep them clean

Testing Types (MVP)

Manual Testing (Developer)

- Login / logout (email + Google OAuth)
- Student cannot access tutor pages (URL hacking test)
- Tutor cannot access student-only pages
- Homework assignment visibility (correct student only)
- Message delivery
- Tuition fee display + password lock
- Outstanding payment calculation
- File attachment upload and download
- Mobile browser usability

Role-Based Permission Testing

Action	Student	Parent	Tutor
View homework	■	■	■
Create homework	■	■	■
Upload attachment	■	■	■
View fees	■	■ (password)	■
Record payment	■	■	■
Manage students	■	■	■
Send messages	■	■	■
Mark homework done	■	■	■
Google OAuth login	■	■	■

User Testing (Realistic)

- Ask 1 parent and 1 student to use the app
- Observe: confusion points, missed buttons, slow actions
- No feedback is too small — all noted for Phase 2

Bug Handling Rules

Type	Action
Critical bugs	Fix before launch — app cannot go live with these
Minor UI bugs	List for Phase 2 — not blockers
Feature creep	Do NOT add features during testing phase

9 Deployment

◆ **UPDATED:** Deployment stack finalised: Render (Laravel app) + Neon (PostgreSQL) + UptimeRobot (keep-alive). Total cost: \$0/month.

Deployment Stack

Component	Service	Cost	Notes
Laravel App	Render	\$0 (free tier)	Auto-deploy from GitHub. Spins down after 15 min idle.
Keep-Alive	UptimeRobot	\$0 (free forever)	Pings app every 5 min. Prevents cold starts. Personal use free.
Database	Neon	\$0 (free tier)	PostgreSQL. 0.5GB. Scale-to-zero. 100 free projects.
Domain	Custom domain	~\$10-15/year (opt.)	Optional. App works on render URL without custom domain.
SSL/HTTPS	Render built-in	\$0	Automatic SSL on Render. No setup needed.

UptimeRobot Setup (Cold Start Prevention)

1. Deploy app to Render → get URL (e.g. mytuitionapp.onrender.com) 2. Sign up at uptimerobot.com (free, no credit card) 3. Add new monitor → HTTP(S) monitor 4. Enter your Render app URL 5. Set check interval: every 5 minutes 6. Done — app stays awake 24/7 for free Result: Render never hits the 15-minute idle limit. No cold starts. \$0 cost.

When to Upgrade Render (\$7/month)

- You have consistent daily users
- Cold start delay starts annoying real students/parents
- You are actively collecting tuition fees and the app is in production
- You need guaranteed uptime

Post-Deployment Checklist

- Login works (email + Google OAuth)
- Correct role redirection (student vs tutor)
- Homework visible to correct student only
- File attachments upload and download correctly
- Fee section hidden from students, unlockable by parent
- Outstanding payment calculation correct
- Messages deliver correctly
- Mobile browser usable
- UptimeRobot pinging correctly

Environment Variables (.env)

```
APP_NAME=Tuition Management App APP_URL=https://yourapp.onrender.com
DB_CONNECTION=pgsql DB_HOST= DB_DATABASE= DB_USERNAME=
DB_PASSWORD= GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret
GOOGLE_REDIRECT_URI=https://yourapp.onrender.com/auth/google/callback
FEE_VIEW_PASSWORD=your-secret-parent-password APP_DEBUG=false ← MUST be
false in production
```

Phase 2 Features (Planned)

- Full marketing website: About Us, Courses page (Primary 1-6 / Secondary 1-5 / Subjects), Contact Us
- Attendance tracking
- Exams & report cards — auto-generated (for tutor portfolio and all parties to track student progress)
- Short Answer Quiz questions with AI marking (Google Gemini free tier primary, Claude Haiku fallback)
- Quiz PDF question extraction — extract questions from exam paper PDFs automatically
- Parent & student separate roles / logins
- Message replies (students can reply to tutor)
- Student self-edit profile
- PWA offline mode (cached pages work without internet)
- Per-student fee unlock password (upgrade from global)
- Push notification toggle settings (user can choose which notifications to receive)

Phase 3 Features (Future)

- Native mobile app (React Native) — only if users genuinely demand it. PWA should cover most needs.
- SMS integration
- Reports & data export (PDF/Excel)
- Backup & restore
- Multi-tenant version for tuition centres
- Subscription plans for tutors (if productising for other tutors)

Business Growth Possibilities

- Multi-tenant version for tuition centres
- Subscription plans for tutors
- White-label for tuition centres
- Staff salary management (if serving centres)

Performance & Maintenance

- Bug fixes post-launch
- UI refinements based on user feedback
- Database optimisation as data grows
- Regular security updates (Laravel + PHP)
- Upgrade Render to paid tier when usage justifies it

Pricing Strategy (Quiz AI Marking Cost)

Recommended pricing: SGD 5 per student per month Cost breakdown (10 students, 2 lessons/week, 10 questions per lesson): Google Gemini API (free tier): Free - primary AI marker Claude Haiku fallback: approx USD 0.001 per question = approx USD 0.08 per student per month Render hosting: Free tier Neon database: Free tier Total costs: approx USD 0.80 per month for 10 students Revenue at SGD 5 per student: SGD 50 per month Net profit: approx SGD 49 per month SGD 5 per student per month is very comfortable. Could even charge SGD 3 and still profit well.

End of Document

Wowlo — Tuition Management App — Full Product Document v2.0